JOHANNES KEPLER
UNIVERSITY LINZ

Simon Grundner
k12136610

Institute for Integrated Circuits and Quantum Computing

k12136610@students.jku.at
https://github.com/s-grundner/LVA-EIC-KV

January 10, 2026

Tiny Tapeout - MIDI Polyphonic Synthesizer

Technical Report

KV Integrated Circuits Design - WiSe25

Abstract MIDI Polyphonic Synthesizer on the Tiny Tapeout ASIC

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Contents

Abstract ii

Acronyms iii

1. Introduction 1

2. Digital Instruments and Music Theory 2
- 2.1 The MIDI-Protocol . . . 2
- 2.2 Instrument Tuning . . . 3
- 2.3 Instrument Characteristics . . . 3
- 2.4 Polyphony . . . 4
- 2.5 MIDI Physical Layer . . . 5

3. System Overview 6
- 3.1 Pinout . . . 6
- 3.2 Block Diagram . . . 6
- 3.3 Layout . . . 7

4. Reciever and MIDI-Decoder 8

5. Synthesizer 9
- 5.1 Frequency Calculation and Generation . . . 9
- 5.2 Oscillatorstack . . . 10

6. System Testing 10

7. External Hardware 10
- 7.1 MIDI-Source . . . 10
- 7.2 Input Side . . . 10
- 7.3 Output Side . . . 10

8. Used Software 11

References 11

Abstract

The project on hand implements a polyphonic audio square wave synthesizer in Verilog, which can be controlled via the musical instrument digital interface (MIDI) protocol. The project is allocated on the TTSKY25b shuttle of the TinyTapeout project and is to be manufactured under the SkyWater 130 nm process. This report documents functionality as well as implementation and discusses design constraints, imposed by the limited physical assigned space on the shuttle.

Acronyms

ASIC application specific integrated circuit

DAC digital to analog converter

DAW digital audio workstation

HDL hardware description language

GM1 general MIDI level 1

GM2 general MIDI level 2

MIDI musical instrument digital interface

PCB printed circuit board

PDK process development kit

UART universal asynchronous reciever transmitter

USB universal serial bus

SPN scientific pitch notation

12ET twelve tone equal temperament

I2S inter-integrated sound

1. Introduction

MIDI is a versatile digital interface, which packs note expressions into a simple protocol. The note expression often origins from a MIDI-controller (source) and contains information about which key on the pianoroll is affected and how it is affected. A key thing to notice is, that the note expression itself does not contain any information about how the audiosignal sounds. The synthesis of the waveform is handled by a MIDI-instrument (sink) and translates the note expression into a audio timesignal. Figure 1 shows how a MIDI-keyboard might interface with a MIDI compatible synthesizer.

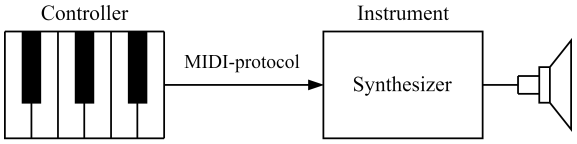

Figure 1: Setup of a controller-instrument-pair communicating via the MIDI-protocol.

The project implements such a MIDI-instrument and converts incoming MIDI-signals into audible squarewaves. The project is written in the hardware description language (HDL) Verilog specifically to be built as a digital application specific integrated circuit (ASIC) design by the open source LibreLane toolchain. The resulting chip will be taped out in collaboration with the Tiny-Tapeout project.

The Tiny-Tapeout project makes manufacturing ASIC-designs for individuals feasable by combining multiple smaller designs on a single chip under the usage of an open source process development kit (PDK). The chip space is sectioned in tiles, where each tile has around $160 \times 100\,\mu m$ of available space. The project on hand occupies one tile. The active run at this time, TTSKY25b, manufactures the chip under SkyWater 130 nm PDK ruleset.

2. Digital Instruments and Music Theory

Before discussing the design, a few fundamentals of music theory have to be considered and how implementing a digital instrument in silicon is sensible. The MIDI Manufactures Association defines the general MIDI level 1 (GM1) and its extension the general MIDI level 2 (GM2) standard to provide guidelines for developing MIDI compatible instruments.

2.1 The MIDI-Protocol

The MIDI-protocol is based on a serial asynchronous data transmission, where at most three bytes are sent in succession, with a dataformat as shown in Table 1. It is noted, that not all commands require a second data byte which is why it is specified in brackets as an optional byte. In this report all required commands provide a second data byte, therefor a length of three bytes will be assumed from now on.

Table 1: Dataformat of a MIDI-protocol transmission

Byte 1		Byte 2	Byte 3
Statusbyte		Databyte 1	[Databyte 2]
Command	Channel		

Table 2: Excerpt from the MIDI 1.0 specification message summary for channel voice messages [4].
$N = nnnn \in [0; 15]$ maps to a MIDI channel number 1-16.
$K = kkkkkkk \in [0; 127]$ is the note number. (e.g. which key on a piano is pressed/released)
$V = vvvvvvv \in [0; 127]$ is the velocity (how fast a note is pressed/released)

Status	Databytes	Description
`1000nnnn`	`0kkkkkkk` `0vvvvvvv`	Note Off event. This message is sent when a note is released (ended).
`1001nnnn`	`0kkkkkkk` `0vvvvvvv`	Note On event. This message is sent when a note is pressed (started).

The first byte combines two four-bit values representing a command and a channel number. Depending on the command code, the following databytes can be interpreted accordingly by a MIDI-sink listening on a pre-set channel. The most important and only command codes used in this project are listed in Table 2. A seven bit note number (denoted as `kkkkkkk` in Table 2) maps to the piano keys of a twelve tone equal temperament (12ET), where the middle C (C_4) is equivalent to a MIDI note number of 60 in decimal [3, sec 2.7.1.1].

2.2 Instrument Tuning

A musical octave is an interval of two notes, where the frequency of the higher note is exactly twice as high as the lower note. The 12ET is a system which divides this octave into twelve notes, which are referred to in scientific pitch notation (SPN) as letters from A to G with accidentals (♭, ♯) and an octave number n as shown in Table 3.

Table 3: SPN Nomenclature for notes in a 12 tone tempered octave, where n is the octave-number.

B_{n-1}	C_n	$\frac{C\sharp_n}{D\flat_n}$	D_n	$\frac{D\sharp_n}{E\flat_n}$	E_n	F_n	$\frac{F\sharp_n}{G\flat_n}$	G_n	$\frac{G\sharp_n}{A\flat_n}$	A_n	$\frac{A\sharp_n}{B\flat_n}$	B_n	C_{n+1}

These notes are equally tempered, meaning the frequency-ratio of two adjacent notes is constant for all musical notes. For an octave of 12 notes, this ratio equates to $\sqrt[12]{2} \approx 1.05946$, leading naturally to a logarithmic scale.

To assert actual frequency values to the notes, a pitch standard has to be decided on. GM2 suggests the ISO-16 pitch standard, where note number 69 in decimal (A_4) is tuned to 440 Hz [3, sec. 2.7.1.1]. With this reference frequency, the absolute frequency of all notes can be calculated by multiplying or dividing by the defined ratio, yielding Equation 1 for the note frequency $f(K)$ in dependence of the MIDI note number K.

$$f(K) = 440\,\text{Hz} \cdot (\sqrt[12]{2}\,)^{(K-69)} = 440\,\text{Hz} \cdot 2^{(K-69)/12} \tag{1}$$

2.3 Instrument Characteristics

The frequency in Equation 1 yields the fundamental frequency of the sound oscillation generated by the instrument. However, the characteristic sound an instrument produces is caused by overtones relative to its fundamental. For example, the Grand Piano [1] in the digital audio workstation (DAW) Ableton Live (see Used Software) is characterised by overtones as seen in Figure 2.

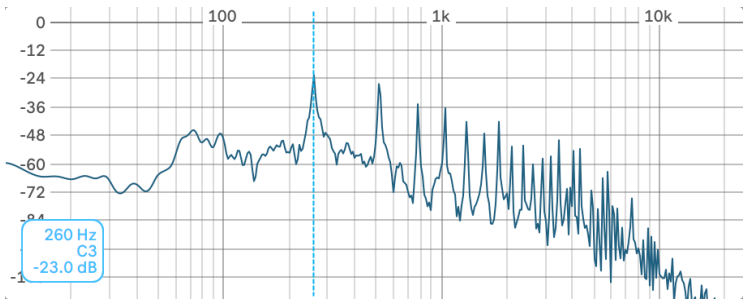

Figure 2: Overtones of a grand piano instrument playing a middle C (C3 in Lives notation).
Abscissa: Audiolevel in dB relative to the fullscale range of the audio engine.
Ortinate: Frequency in Hz.

A classical method to characterise the voice of a synthesizer is by using a wavetable. A wavetable is a discrete list of values representing the waveform amplitude over the span of one period (Figure 3). To synthesize the audio waveform, the table is being iterated at a certain speed, such that one loop spans over the period of the fundamental frequency.

To implement this in the design, the output capabilites of the ASIC have to be considered. As the design is purely digital, the output levels of the chip can only assume the logic levels high and low, which leaves following options:

1. Feed an audio stream to an external DAC using serial connectivity such as I2S.
2. Use all output pins as a parallel interface to drive a parallel DAC asynchronously.
3. Have the digital output be the synthesized waveform.

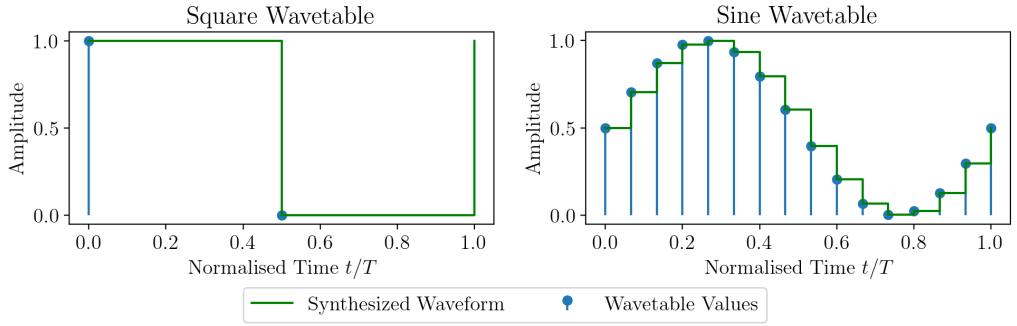

Figure 3: Wavetables for a 1-bit squarewave (left) and a 16-bit sinewave (right)

For this design the latter option is chosen, which has several benefits. Using a single digital output directly as the audio waveform is equivalent to a 1-bit wavetable, resulting in a squarewave. For a 1-bit wavetable the output can simply be toggled every half period.

The spectrum of an ideal squarewave with infinite runtime is defined by a fourier series in Equation 2 containing odd multiples

$$\omega_n = \frac{2\pi}{T} n, \qquad n = 1, 3, 5, \ldots$$

of the fundamental frequency ω_1. The amplitude for harmonics of higher order declines hyperbolically [5], leading to a spectrum as shown in Figure 4.

$$s(t) := \sum_{n=1,3,5,\ldots}^{\infty} \frac{4}{n\pi} \sin(\omega_n t) \quad \overset{\|\cdot\|}{\circ\!\!-\!\!-} \quad \|S(j\omega)\| := \sum_{n=1,3,5,\ldots}^{\infty} \frac{4}{n} \delta(\omega - \omega_n) \tag{2}$$

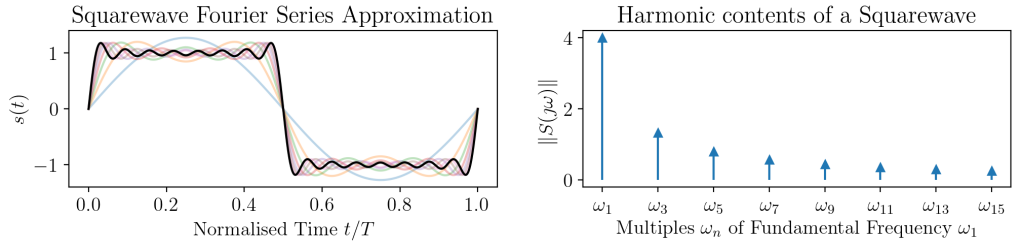

Figure 4: Approximation of a squarewave by adding odd harmonics one by one

2.4 Polyphony

It is common for instruments to play two notes simultaneously, which is caused by receiving multiple note-on commands without turning already active MIDI notes off. An important

parameter is the number of voices the instrument can produce. An instrument is considered polyphonic, if the number of voices is at least two. In this case, each voice is represented by a individual squarewave oscillators. One of the main challenges in this design is to reduce the area for the implementation of a voice oscillator to achieve a high polyphony count.

The audiosignal is directly sent to a digital pin. However, overlapping squarewaves of differing phase and frequency, again produce an analog signal (shown in Figure 5), which cannot be output by a single pin. Therefor, each voice has its own output pin. The individual voices are to be summed together by external circuitry, for example by an operational amplifier circuit (see section 7.3).

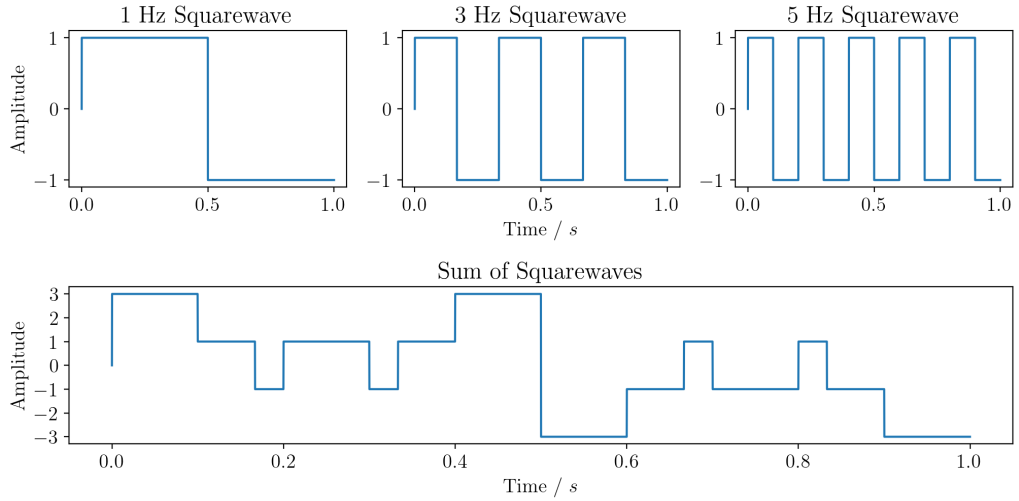

Figure 5: Individual Squarewaves summed up to an analog signal

2.5 MIDI Physical Layer

The MIDI protocol was originally transmitted asynchronously over a differential pair, which was daisychained between MIDI-controllers and connected with 5 pole DIN connectors. With rising popularity of the universal serial bus (USB), MIDI controllers adapted to this standard. Nowadays most MIDI controllers communicate via USB to a host device. Therefor MIDI instruments can also be controlled via USB under the usage of a USB to UART serial interface as it is intended with this design.

3. System Overview

The source code can be found under the follwing link:
https://github.com/s-grundner/LVA-EIC-KV

3.1 Pinout

The project features four voices implemented as individual squarewaves oscillators. The output of each oscillator is routed to the outputpins `uo[0]`…`uo[3]`. Output `uo[7]` yields a PWM Signal, which encodes the number of actively running oscillators. This output may be used to control the gain of external circuitry when mixing the oscillator outputs together. The project has one input `ui[3]` to which the MIDI signal has to be supplied to. This pin is also connected to the UART-TX output of the RP2040 Microcrontroller, which is populated on the TinyTapeout test PCB. The RP2040 may be used to emulate a MIDI controller or to interface the instrument via USB. Figure 6 summarises the in- and outputs of the ASIC.

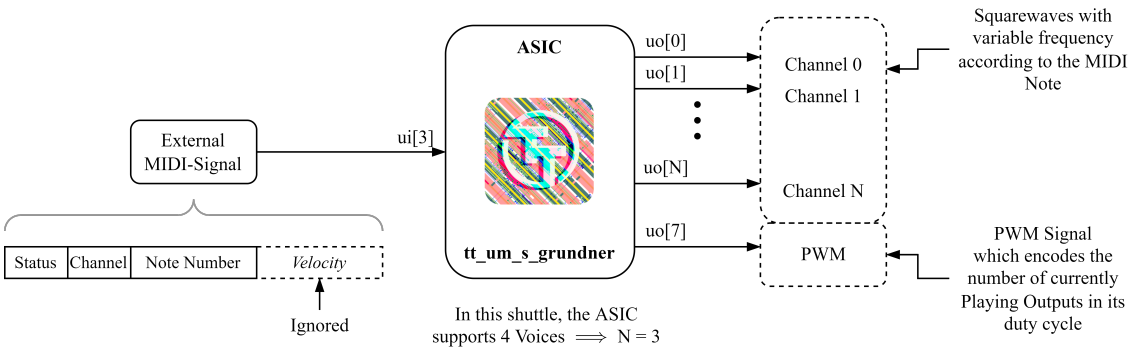

Figure 6: Input and output visualisation and description

3.2 Block Diagram

An abstract overview of the project implementation can be seen in Figure 7. The block-titles outside of each block repesent the Verilog module name coherent with the code-repository.

The signal flow starts at the input, where the MIDI messages are received via UART. The bits of the input stream are collected into bytes, which are then fed into a MIDI decoder. The MIDI decoder extracts the note- and channel number and if a on or off command is received.

The note number indexes a lookup table which returns a counterbase for the corresponding frequency. Each oscillator in the stack is listening on its own MIDI-channel. The note-on/note-off commands only affect the oscillator listening on this channel. Upon recieving a note on command, the counterbase value at the input of an oscillator is latched and a

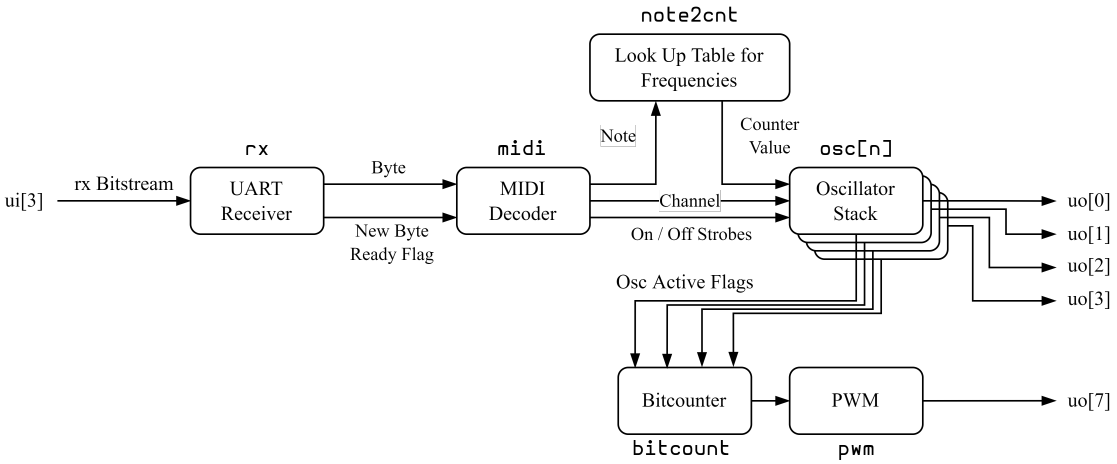

Figure 7: Abstract Blockdiagram of the Synthesizer (`synth` module)

counter starts counting repeatedly in this time-base. The output of the oscillator is then toggled at each counter overflow, which generates the squarewave output.

3.3 Layout

To demonstrate the density of the layout, an image of the generated GDS is shown in Figure 8.

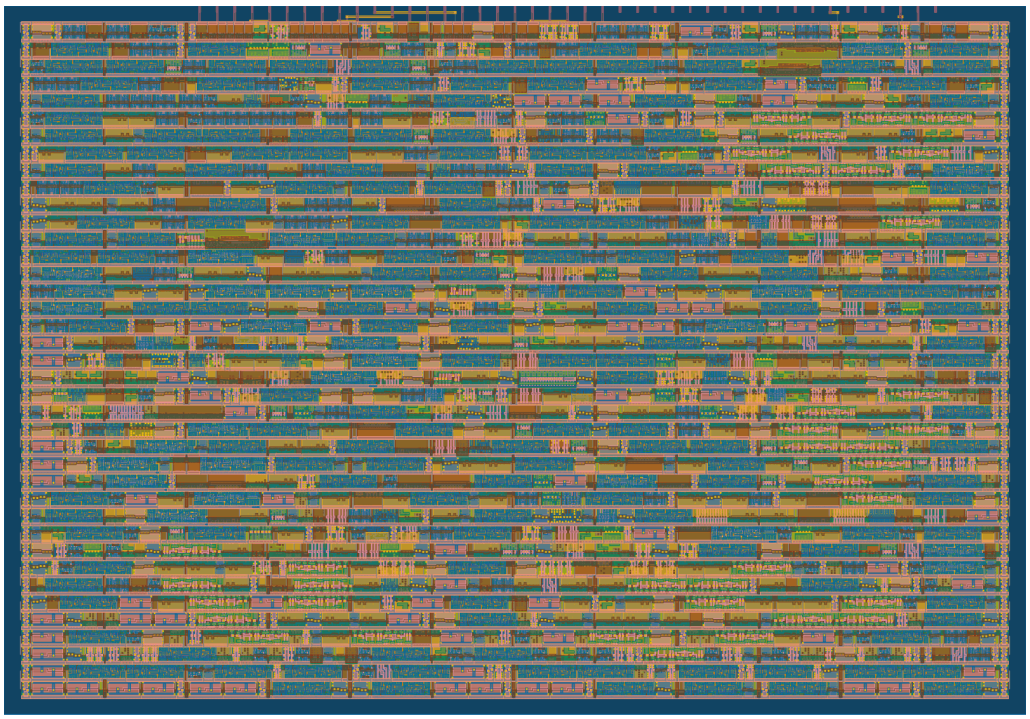

Figure 8: Layout of the Tile

The routing statistics generated by the TinyTapeout pipeline are listed in Table 4. Especially a space utilisation of 74.275 % shows, that space-optimization was crucial. Per

default, the maximum density is set to 60 %, which had to be adjusted to fit more features. The documentation of the TinyTapeout project stated, that a density of up to 80 % worked in previous shuttles without issues. The layout as it is shown in Figure 8 is comprised of logic cells which are predefined by the PDK. Which logic cells are generated from the Verilog code can be seen in Table 5.

Table 4: Routing statistics

Utilisation / %	Wire length / µm
74.275	23342

Table 5: Used Logic cells of the PDK and the number of occurences

Category	Cells	Count
Fill	decap fill	733
Combo Logic	a22o a41o and2b a2bb2o o22ai a31o a22oi o21ai a221o o22a o2111ai a211o or4b a21oi a21o o211a and3b a311o o21a o2bb2a o221a a2111o and4b o211ai or4bb a211oi or3b a31oi o41ai o21ba a21bo a32o a221oi o2111a and4bb o32a	228
Tap	tapvpwrvgnd	225
Flip Flops	dfrtp dfstp	191
Multiplexer	mux4 mux2	180
Misc	conb dlymetal6s2s dlygate4sd3	98
AND	and3 and2 and4 a21boi	95
Buffer	clkbuf buf	79
NOR	nor2 xnor2	59
Inverter	inv	49
OR	or2 xor2 or3 or4	48
NAND	nand2 nand3 nand2b	32
Clock	clkinv	10
1069 total cells (excluding fill and tap cells)		

4. Reciever and MIDI-Decoder

uart

MIDI Statemachine. has four states. Switches states on new byte ready. Velocity byte is required for the statemachine to switch states correctly, but the value is ignored. A way to interpret velocity, would be to reduce the volume of the output. As the Synthesizer only has one digital amplitude, this is obsolete.

Voice Stealing

MIDI Channel routing

Limitations: According to the General MIDI-System Level 1 (GM-1) Specifications, a Sound Generating Instrument must implement a set of features to ensure compatibility with various controller outputs / midi-sequences [2]. This Project does not comply with GM-1 as even the smallest feature-set requires a much more complex midi statemachine which is beyond the goals of this project.

5. Synthesizer

How is the Frequency Generated?

Count up a defined Number of steps

For each Count, a time interval of 1/fclk elapses.

The Lowest Frequency requires the highest number of counts

The Task is to (space) efficiently determine the Counter Period from the MIDI-Note number

5.1 Frequency Calculation and Generation

Limitations:

- Frequency Resolution determined by Clock Frequency - Divisions and Modulo operations are Expensive.

Choosing a Clock Frequency

- See reasoning in Py Notebook

Externalize Calculation Logic from the Oscillator Stack

Compare optimisations

- Performing an actual Twelvetone Temparament Calculation on Hardware

- Lookup Table: Save the Countervalues of the Twelve Notes on the Octave to achieve Most precise Counterperiod. What Calculations are required by the oscillators

- Lookup Table: Reduce Bitwitdth and therefor required LUT Size. Compare Size Reduction

Impact of Frequency Deviation

- Systematic error of frequencies from reducing counter resolution. Deviations and calculations from Py-Notebook.

- Spectrum and Harmonics

Are the Deviations Audible?

- Heavily Dependent on the individual Person (Hearing Curve)

- Document Experiment in Ableton Live

5.2 Oscillatorstack

Routing a MIDI a Channel to a fixed Oscillator.

Modularity by a Generating Function: Option to increase number of voices for larger space or future Design optimisations.

6. System Testing

Edge Cases

Limitations. Which tests are omitted. Correctness of the midi Input Bytoe Order for example.

CocoTB Testresults, Design Simulation+validation, Measurements, Waveforms

7. External Hardware

7.1 MIDI-Source

As the device does not fully comply to the GM-1 specs, it is best to provide MIDI inputs from a controlled environment. Pluggin in a full-fledged midi controller directly might lead to unwanted behaviour.

Example of a USB-MIDI controller.

In Principle Direct connection is Possible but a Powersupply is required.

Setting up a Virtual MIDI-Source with a DAW, Hairless MIDI and

PMOD as a PHY for the native Differential midi connection.

7.2 Input Side

Digital Frontend converting simultaneous Notes into individual Channel

7.3 Output Side

Audio Mixing Circuit, Gain Control, Speaker Driver

Option for mild subtractive Sound-Design

- Squarewaves provide all odd harmonics of the fundamental frequencies, which can be filtered by an external system to create different Sounddesigns

- optimal Would be a Sawtooth as it contains all harmonics but violated the digitality of the ASIC Output.

Table 6: List of used software for development and comissioning this project

Name	Version	Description	Usage
Ableton Live 11 Suite	11.3.43	DAW	

8. Used Software

appendix: Setting up a development env.

References

[1] Ableton AG. Live Grand Piano. https://www.ableton.com/en/packs/grand-piano. Accessed: 2026-01-02.

[2] The MIDI Manufacturers Association. General MIDI System Level 1 Specification. https://midi.org/general-midi-level-1, 1996. Accessed: 2025-12-25.

[3] The MIDI Manufacturers Association. General MIDI System Level 2 Specification. https://midi.org/general-midi-level-2-2, 2007. Accessed: 2025-12-25.

[4] The MIDI Manufacturers Association. Summary of MIDI 1.0 Messages. https://midi.org/summary-of-midi-1-0-messages, 2020. Accessed: 2025-12-25.

[5] Weisstein, Eric W. Fourier Series–Square Wave. From MathWorld–A Wolfram Resource. https://mathworld.wolfram.com/FourierSeriesSquareWave.html. Accessed: 2026-01-02.

Wikipedia References

W MIDI tuning standard - Wikipedia

W Scientific pitch notation - Wikipedia

W 12 equal temperament - Wikipedia

W A440 (pitch standard) - Wikipedia